Reporte Técnico de Actividades Práctico-Experimentales Nro 3

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Anthony Yaguana
Asignatura	Simulación
Ciclo	5
Unidad	1
Resultado de aprendizaje de la unidad	**R1.** Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Simulación de un sistema de líneas de espera M/M/1 mediante SimPy
Nombre del Docente	José O. Guamán Q.
Fecha	15 de Octubre de 2026 16 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Simular un sistema de líneas de espera M/M/1 utilizando Python y SimPy, analizando el comportamiento de los clientes, la cola y el servidor.

- Implementar un modelo M/M/1 utilizando la biblioteca SimPy.
- Calcular las principales métricas teóricas del sistema de espera.
- Comparar los resultados teóricos con los obtenidos mediante la simulación.

3. Materiales, Reactivos, Equipos y Herramientas

- Python 3.x.
- Biblioteca SimPy.
- NumPy.
- Matplotlib.
- Editor de código: Visual Studio Code, PyCharm o Jupyter Notebook.
- Computador.
- Procesador de 2 núcleos o superior.

- Mínimo 4 GB de RAM.
- Sistema operativo Linux o macOS

4. Procedimiento / Metodología Ejecutada

Fundamento teórico

Un sistema de líneas de espera está formado por clientes que llegan a solicitar un servicio, una cola en la que esperan y uno o más servidores que los atienden [1]. En la notación de Kendall, el sistema M/M/1 tiene tiempos entre llegadas exponenciales (primera M), tiempos de servicio exponenciales (segunda M) y un único servidor (1). En este trabajo la disciplina de atención es FIFO, es decir, el primer cliente en llegar es el primero en ser atendido, y la cola no tiene límite de capacidad [1], [2].

La letra M significa «markoviano». Las llegadas forman un proceso de Poisson con tasa λ, por lo que el tiempo entre dos llegadas consecutivas sigue una distribución exponencial con media $1/\lambda$; del mismo modo, la atención de cada cliente dura un tiempo exponencial con media $1/\mu$ [2]. La distribución exponencial carece de memoria: la probabilidad de que un evento ocurra en el instante siguiente no depende del tiempo que ya transcurrió, y gracias a esa propiedad el sistema puede modelarse como un proceso de Markov [3].

El parámetro central del modelo es la intensidad de tráfico o utilización del servidor, que representa la fracción del tiempo en que el servidor permanece ocupado [2]. El sistema alcanza un estado estacionario solo si la utilización es menor que 1, es decir, si los clientes llegan más despacio de lo que el servidor puede atenderlos; en caso contrario la cola crece sin límite [1]:

$$\rho = \lambda / \mu < 1 \quad (1)$$

En estado estacionario, las medidas de desempeño del modelo M/M/1 se obtienen con las siguientes expresiones [1], [2]:

$$L = \lambda / (\mu - \lambda) \quad (2) \text{ clientes promedio en el sistema}$$

$$Lq = \lambda^2 / (\mu(\mu - \lambda)) \quad (3) \text{ clientes promedio en la cola}$$

$$W = 1 / (\mu - \lambda) \quad (4) \text{ tiempo promedio en el sistema}$$

$$Wq = \lambda / (\mu(\mu - \lambda)) \quad (5) \text{ tiempo promedio de espera en la cola}$$

Estas medidas se relacionan mediante la ley de Little, $L = \lambda W$ y $Lq = \lambda Wq$, que se cumple en cualquier sistema de colas estable [4]. Además, $W = Wq + 1/\mu$, porque el tiempo total que un cliente pasa en el sistema es su espera en la cola más su tiempo de servicio.

Para estudiar el sistema sin depender únicamente de las fórmulas se utiliza la simulación de eventos discretos: el estado del sistema cambia solo en instantes concretos, como una llegada o el fin de un servicio, y el reloj de la simulación avanza de un evento al siguiente [3]. SimPy es una biblioteca de Python para este tipo de simulación basada en procesos: cada cliente se representa como un proceso que espera eventos y el servidor como un recurso de capacidad limitada [5].

Finalmente, para estimar el tamaño promedio de la cola a partir de la simulación puede usarse lo que observa cada cliente al llegar, ya que las llegadas de Poisson

observan el sistema en su promedio temporal (propiedad PASTA, Poisson Arrivals See Time Averages) [6].

La tabla 1 resume la notación utilizada en este informe.

Símbolo	Significado	Expresión
λ	Tasa promedio de llegada de clientes	clientes / unidad de tiempo
μ	Tasa promedio de servicio del servidor	clientes / unidad de tiempo
ρ	Utilización del servidor	λ / μ
L	Número promedio de clientes en el sistema	$\lambda / (\mu - \lambda)$
Lq	Número promedio de clientes en la cola	$\lambda^2 / (\mu(\mu - \lambda))$
W	Tiempo promedio de un cliente en el sistema	$1 / (\mu - \lambda)$
Wq	Tiempo promedio de espera en la cola	$\lambda / (\mu(\mu - \lambda))$

Tabla 1. Notación del modelo M/M/1

Metodología ejecutada

La práctica se desarrolló con Python 3.12 con SimPy, NumPy y Matplotlib, siguiendo estas etapas:

- **Análisis de la guía.** Se revisaron el modelo M/M/1, sus fórmulas, el pseudocódigo y los tres escenarios propuestos en la guía de la práctica. En este paso se detectó que la fórmula de la utilización debía ser $\rho = \lambda/\mu$.
- **Creación del repositorio.** Se creó el repositorio en GitHub (github.com/ivecanahi/ape3) con la estructura base del proyecto: carpetas src/, tests/ y resultados/, y el archivo requirements.txt.
- **División del trabajo por módulos.** El proyecto se dividió en cuatro partes independientes (teoría, simulación, gráficas e integración). Antes de programar se acordaron los nombres de las funciones y de los datos que cada parte entrega, para poder avanzar en paralelo.
- **Trabajo por ramas.** Cada parte se desarrolló en una rama propia de Git y se integró a la rama principal (main) mediante Pull Requests.
- **Desarrollo con pruebas.** Las fórmulas teóricas se programaron escribiendo primero las pruebas con los valores esperados de cada escenario y luego el código. En total, el proyecto tiene 17 pruebas automatizadas (pytest).
- **Integración.** Se unieron la teoría, la simulación y las gráficas en un programa principal (main.py) con un menú que permite elegir qué escenario simular o ejecutar los tres con su comparación.
- **Revisión y corrección.** Al revisar el repositorio se detectó que el tamaño de la cola solo se registraba cuando llegaba un cliente. Se corrigió para registrarlo también cuando un cliente pasa a ser atendido y se regeneraron las gráficas. También se mejoró la presentación de los resultados en la consola.
- **Ejecución y análisis.** Se ejecutaron los tres escenarios, se compararon los resultados teóricos con los simulados y se analizaron las gráficas.

El proyecto quedó organizado en los siguientes módulos:

Módulo	Función
src/teoria.py	Calcula ρ, L, Lq, W y Wq y valida la estabilidad (λ < μ).
src/simulacion.py	Simula la cola M/M/1 con SimPy y registra los tiempos de espera y el tamaño de la cola.
src/graficas.py	Genera las gráficas de la cola, de los tiempos de espera y la comparación.
src/experimentos.py	Define los escenarios, une teoría y simulación y genera la tabla comparativa.
src/consola.py	Presenta los resultados en la consola con formato (biblioteca rich).
main.py	Programa principal (python main.py): muestra un menú para elegir el escenario que se simula.

Tabla 2. Módulos del proyecto

La simulación, que sigue el pseudocódigo de la guía, funciona así:

- Se definieron λ, μ, el tiempo de simulación (10 000 unidades de tiempo) y una semilla fija (42) para que los resultados sean reproducibles.
- Antes de simular se verificó la condición de estabilidad λ < μ; si no se cumple, el programa muestra el mensaje «El sistema no es estable».
- Se creó el entorno de SimPy (simpy.Environment) y un servidor con capacidad 1 (simpy.Resource) [5].
- Un proceso generador crea clientes con tiempos entre llegadas exponenciales de media 1/λ.
- Cada cliente registra su llegada, solicita el servidor, espera su turno en orden de llegada (FIFO), registra su tiempo de espera y es atendido durante un tiempo exponencial de media 1/μ; al terminar libera el servidor.
- El tamaño de la cola se registró cuando llega cada cliente y cuando cada cliente pasa a ser atendido. El tamaño promedio de la cola simulado es el promedio de la cola observada por los clientes al llegar; por la propiedad PASTA (las llegadas de Poisson observan el promedio temporal del sistema), es un estimador válido de Lq [6].
- Al finalizar se calcularon los promedios simulados, las métricas teóricas y el error porcentual |teórico − simulado| / teórico × 100, y se generaron la tabla comparativa y las gráficas.

Se ejecutaron los tres escenarios propuestos en la guía:

Escenario	λ	μ	ρ = λ/μ	Condición
1	2.0	3.0	0.667	Sistema estable
2	2.5	3.0	0.833	Mayor utilización
3	2.9	3.0	0.967	Cerca de saturación

Tabla 3. Escenarios de simulación

Para reproducir los resultados se ejecuta *pip install -r requirements.txt* y luego *python main.py*. El programa muestra un menú: las opciones 1, 2 y 3 simulan un

solo escenario, la opción 4 simula los tres y genera la comparación, y la opción 0 termina el programa. Los resultados se imprimen en la consola y las gráficas y la tabla se guardan en la carpeta *resultados/*.

5. Resultados

En esta sección se presentan los resultados obtenidos al ejecutar el programa con los tres escenarios de la tabla 3. Todos los escenarios comparten la misma tasa de servicio (μ = 3.0 clientes por unidad de tiempo) y solo cambia la tasa de llegada λ, de modo que es posible observar cómo responde el sistema a medida que la utilización del servidor se acerca a 1.

Cada escenario se simuló durante 10 000 unidades de tiempo con la semilla 42, lo que equivale a atender entre 19 997 y 28 906 clientes según el escenario. Con esos datos se calcularon el tiempo promedio de espera en cola y el tamaño promedio de la cola, que se comparan con los valores teóricos Wq y Lq de las ecuaciones (3) y (5). La diferencia entre ambos se expresa como error porcentual, calculado como |teórico − simulado| / teórico × 100.

Primero se muestran las métricas teóricas (tabla 4) y la comparación con la simulación (tabla 5 y figura 1). Después se presentan las gráficas del comportamiento de la cola en el tiempo (figuras 2 a 4) y la distribución de los tiempos de espera (figuras 5 a 7). La sección termina con el análisis de los resultados.

Escenario	λ	μ	ρ	L	Lq	W	Wq
1	2.0	3.0	0.667	2.000	1.333	1.000	0.667
2	2.5	3.0	0.833	5.000	4.167	2.000	1.667
3	2.9	3.0	0.967	29.000	28.033	10.000	9.667

Tabla 4. Métricas teóricas del modelo M/M/1

Esc.	Clientes atendidos	Wq teórico	Wq simulado	Error Wq	Lq teórico	Lq simulado	Error Lq
1	19 997	0.667	0.705	5.8 %	1.333	1.398	4.8 %
2	25 008	1.667	1.799	7.9 %	4.167	4.490	7.8 %
3	28 906	9.667	8.228	14.9 %	28.033	23.758	15.2 %

Tabla 5. Comparación entre resultados teóricos y simulados

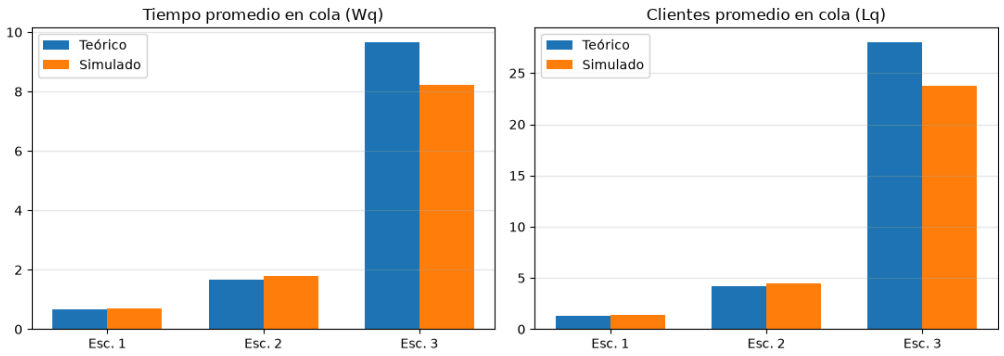


Figura 1. Comparación teórico vs. simulado de Wq y Lq en los tres escenarios

Comportamiento de la cola

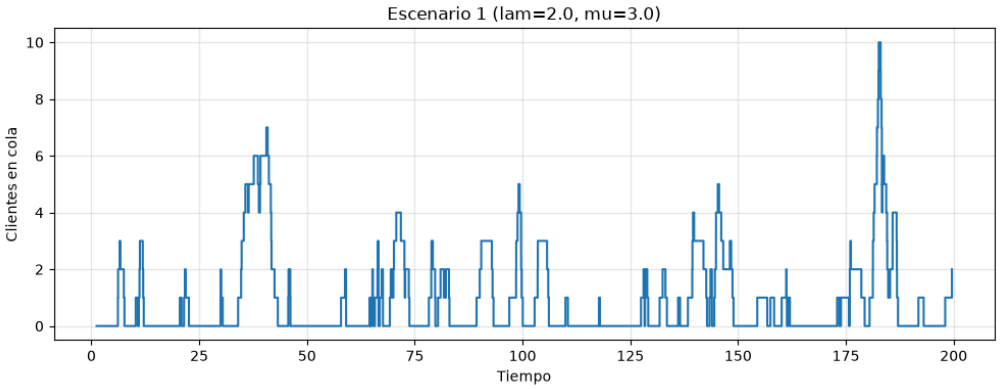


Figura 2. Clientes en cola durante las primeras 200 unidades de tiempo, escenario 1

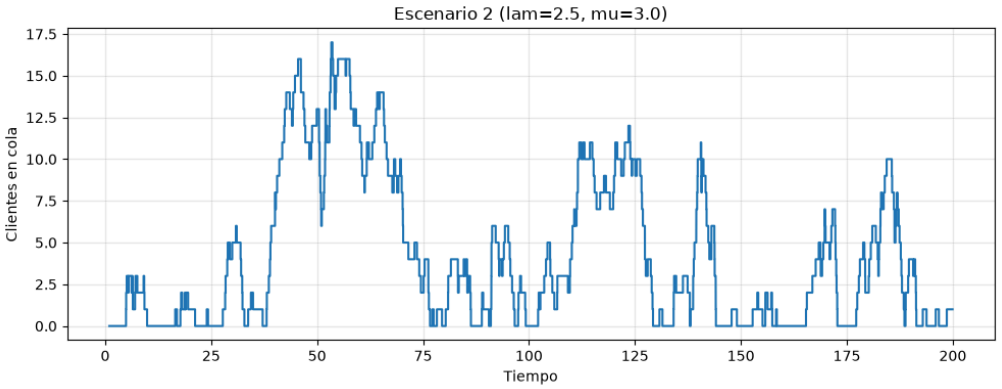


Figura 3. Clientes en cola durante las primeras 200 unidades de tiempo, escenario 2

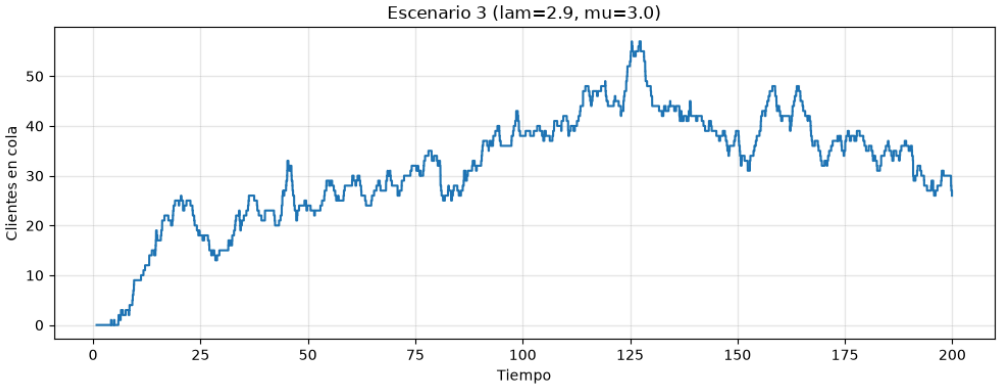


Figura 4. Clientes en cola durante las primeras 200 unidades de tiempo, escenario 3

Distribución de los tiempos de espera

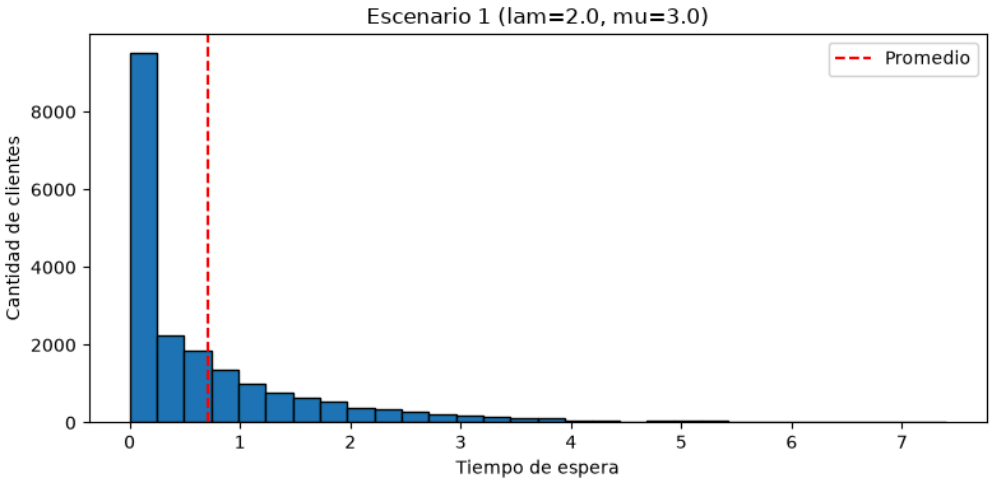


Figura 5. Histograma de tiempos de espera, escenario 1

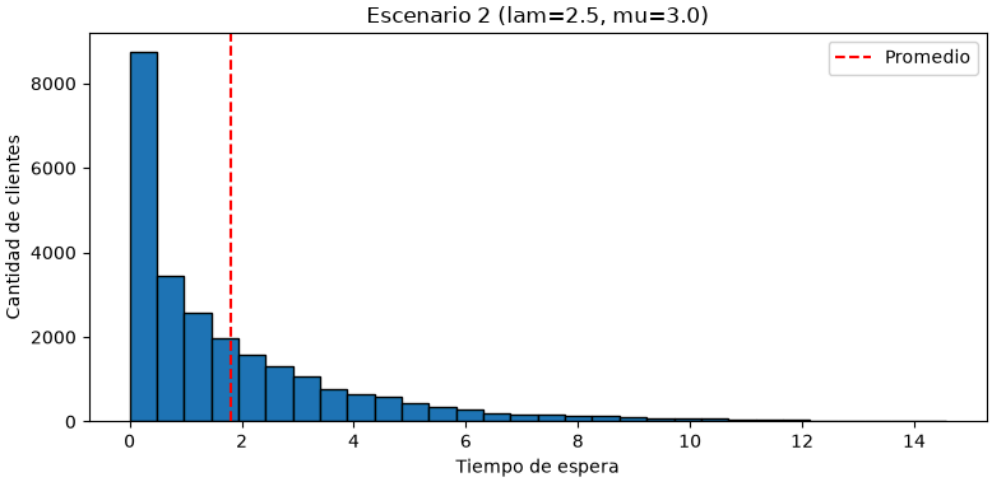


Figura 6. Histograma de tiempos de espera, escenario 2

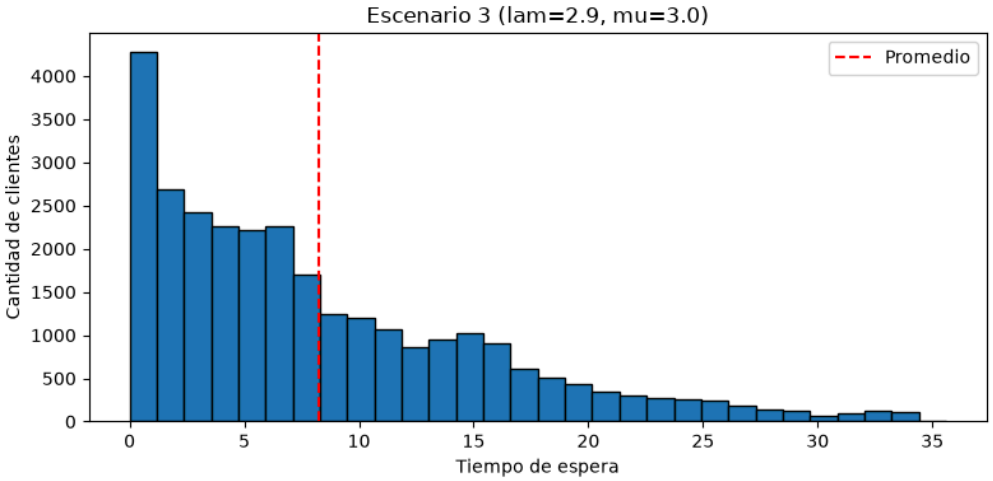


Figura 7. Histograma de tiempos de espera, escenario 3

6. Preguntas de Control

- **¿Qué representa el modelo M/M/1?**

  Es un sistema de una línea de espera con un único servidor, llegadas de Poisson y tiempos de servicio exponenciales, los clientes se atienden en orden de llegada usando FIFO y la cola no tiene límite de capacidad. Sirve para predecir cuánto esperan los clientes y cuán larga se vuelve la cola.

- **¿Qué representan λ y μ?**

  λ es la tasa promedio de llegada de los clientes y μ es la tasa promedio de servicio que da el servidor, ambas en clientes por unidad de tiempo. Sus inversas son los tiempos medios: 1/λ entre llegadas consecutivas y 1/μ de atención por cliente.

- **¿Por qué debe cumplirse λ < μ para que el sistema sea estable?**

  Si los clientes llegan más rápido de lo que el servidor puede atenderlos, la cola acumula clientes sin el tiempo necesario para vaciarse. Solo cuando λ < μ el servidor tiene un tiempo libre y el sistema alcanza un estado estacionario con promedios finitos.

- **¿Qué significa la utilización ρ?**

  Es la fracción del tiempo que el servidor permanece ocupado, calculada como ρ = λ/μ. En los escenarios simulados fue 0.667, 0.833 y 0.967, es decir, el servidor estuvo ocupado en un 67 %, 83 % y 97 % del tiempo.

- **¿Cuál es la diferencia entre W y Wq?**

  Wq es el tiempo que un cliente espera en la cola antes de ser atendido. W es el tiempo total que está en el sistema, es decir, la espera más el servicio (W = Wq + 1/μ).

- **¿Cuál es la diferencia entre L y Lq?**

  Lq cuenta solo a los clientes que esperan en la cola, mientras que L incluye también al cliente que está siendo atendido, por eso L siempre será mayor que Lq.

- **¿Por qué SimPy utiliza una distribución exponencial para generar los tiempos?**

  SimPy no la impone, la exige el mismo modelo M/M/1, y los valores aleatorios se generan con NumPy (rng.exponential). La exponencial es la distribución de los tiempos entre llegadas de un proceso de Poisson y no tiene memoria, lo que permite tratar el sistema como un proceso de Markov.

- **¿Qué sucede con la cola cuando λ se aproxima a μ?**

  La espera y la cola crecen de forma no lineal.

  Al pasar de ρ = 0.667 a 0.967, Lq teórico sube de 1.333 a 28.033 y Wq de 0.667 a 9.667, y aunque λ solo aumentó un 45 %. Las gráficas lo muestran: en el escenario 3 la cola ya no vuelve a cero, mientras que en el escenario 1 se vacía continuamente.

- **¿Por qué los resultados simulados pueden ser diferentes de los resultados teóricos?**

  Las fórmulas describen el promedio de largo plazo (estado estacionario), mientras que la simulación es una muestra aleatoria finita. Influyen el azar de una sola corrida, el arranque con la cola vacía, que subestima los promedios, y los clientes que quedan sin atender al terminar la simulación, por eso el error fue de 5.8 % en el escenario 1 y de 14.9 % en el 3.

- **¿Qué ocurriría si λ fuera mayor que μ?**

  El sistema sería inestable, la cola crecería sin límite con el tiempo y la espera de cada nuevo cliente sería mayor que la del anterior cliente.

  El servidor estaría a full casi todo el tiempo y las fórmulas dejarían de ser válidas, porque darían valores negativos. Por eso el mismo programa de simulación desarrollado lo detecta antes de simular y muestra: “El sistema no es estable”.

7. Conclusiones

- El modelo M/M/1 se implementó correctamente con SimPy, y las métricas simuladas siguieron a las teóricas en los tres escenarios, con errores de 5.8 %, 7.9 % y 14.9 % en Wq, y de 4.8 %, 7.8 % y 15.2 % en Lq. Esto se valida tanto en el código como en las fórmulas.
- Al acercarse λ a μ, el desempeño del sistema se satura de forma desproporcionada y un servidor casi saturado produce esperas muy largas aunque en promedio alcance a atender a todos.
- La diferencia entre la teoría y la simulación aumenta con la utilización, por lo que los resultados de una sola corrida deben interpretarse con mucho cuidado.

8. Declaración de uso de la IA

En la elaboración de esta práctica se utilizó la herramienta de inteligencia artificial Claude, modelo Claude Opus 5.5, desarrollado por Anthropic, para mejorar la redacción técnica del informe y como apoyo en algunas partes del código: la presentación de los resultados en la consola, la corrección del registro del tamaño de la cola y el menú para elegir el escenario. El resto del desarrollo fue realizado por los integrantes. Todo el contenido fue revisado, verificado y ajustado por la autora, quien asume la responsabilidad sobre la versión final.

Herramienta: Claude Opus 5.5 (Anthropic).

Uso: mejora de la redacción técnica del informe y apoyo puntual en partes del código.

Prompts utilizados:

1. Redacción del informe: *«Mejora la redacción técnica de este informe de la práctica 3 de Simulación (sistema de colas M/M/1 con SimPy), manteniendo el contenido y los resultados que obtuvimos.»*

2. Presentación en consola: *«Haz que los resultados de la simulación se vean mejor en la consola.»*

3. Corrección del código: *«Corrige el registro del tamaño de la cola para que también se guarde cuando un cliente pasa a ser atendido.»*

4. Menú de escenarios: *«Haz un menú para poder escoger qué escenario se quiere simular.»*

9. Bibliografía / Referencias

[1] H. A. Taha, *Investigación de operaciones*, 9.ª ed. México: Pearson Educación, 2012.

[2] F. S. Hillier y G. J. Lieberman, *Introducción a la investigación de operaciones*, 9.ª ed. México: McGraw-Hill, 2010.

[3] J. Banks, J. S. Carson, B. L. Nelson y D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[4] J. D. C. Little, "A proof for the queuing formula: L = λW," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[5] Team SimPy, "SimPy: Discrete event simulation for Python," *SimPy Documentation*. [En línea]. Disponible en: https://simpy.readthedocs.io/

[6] R. W. Wolff, "Poisson arrivals see time averages," *Operations Research*, vol. 30, no. 2, pp. 223–231, 1982.

[7] D. G. Kendall, "Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain," *The Annals of Mathematical Statistics*, vol. 24, no. 3, pp. 338–354, 1953, doi: 10.1214/aoms/1177728975.

[8] A. K. Erlang, "Sandsynlighedsregning og telefonsamtaler" [Probability calculation and telephone conversations], *Nyt Tidsskrift for Matematik B*, vol. 20, pp. 33–39, 1909.

[9] D. V. Lindley, "The theory of queues with a single server," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 48, no. 2, pp. 277–289, 1952, doi: 10.1017/S0305004100027638.

[10] J. F. C. Kingman, "The single server queue in heavy traffic," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 57, no. 4, pp. 902–904, 1961.

[11] W. Whitt, "Planning queueing simulations," *Management Science*, vol. 35, no. 11, pp. 1341–1366, 1989, doi: 10.1287/mnsc.35.11.1341.

10. Anexos

- **Repositorio:** https://github.com/Anthony-YP/Simulacion.git
- **Documento:** APE3_U1_SimulacionA.docx